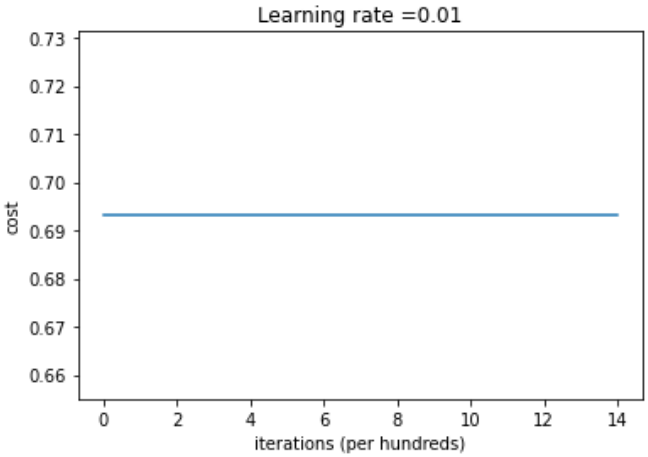
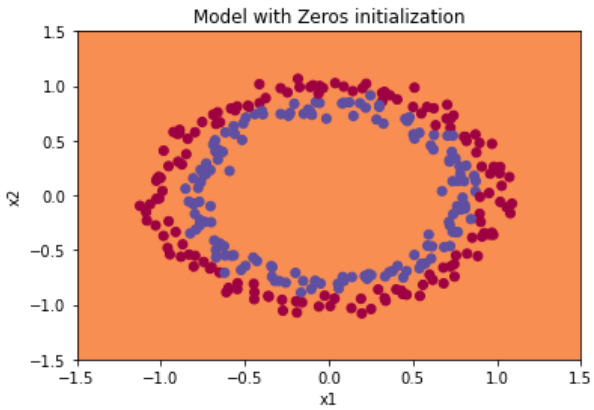
Initialization

```python
def initialize_parameters_zeros(layers_dims):
    """
    Arguments:
    layer_dims -- python array (list) containing the size of each layer.

    Returns:
    parameters -- python dictionary containing your parameters "W1", "b1", ..., "WL", "bL":
                    W1 -- weight matrix of shape (layers_dims[1], layers_dims[0])
                    b1 -- bias vector of shape (layers_dims[1], 1)
                    ...
                    WL -- weight matrix of shape (layers_dims[L], layers_dims[L-1])
                    bL -- bias vector of shape (layers_dims[L], 1)
    """

    parameters = {}
    L = len(layers_dims)            # number of layers in the network

    for l in range(1, L):
        ### START CODE HERE ### (≈ 2 lines of code)
        parameters['W' + str(l)] = np.zeros((layers_dims[l],layers_dims[l-1]))
        parameters['b' + str(l)] = np.zeros((layers_dims[l],1))
        ### END CODE HERE ###
    return parameters
```

```
Cost after iteration 0: 0.6931471805599453
Cost after iteration 1000: 0.6931471805599453
Cost after iteration 2000: 0.6931471805599453
Cost after iteration 3000: 0.6931471805599453
Cost after iteration 4000: 0.6931471805599453
Cost after iteration 5000: 0.6931471805599453
Cost after iteration 6000: 0.6931471805599453
Cost after iteration 7000: 0.6931471805599453
Cost after iteration 8000: 0.6931471805599453
Cost after iteration 9000: 0.6931471805599453
Cost after iteration 10000: 0.6931471805599455
Cost after iteration 11000: 0.6931471805599453
Cost after iteration 12000: 0.6931471805599453
Cost after iteration 13000: 0.6931471805599453
Cost after iteration 14000: 0.6931471805599453
```

```
W1 = [[0. 0. 0.]
 [0. 0. 0.]]
b1 = [[0.]
 [0.]]
W2 = [[0. 0.]]
b2 = [[0.]]
```





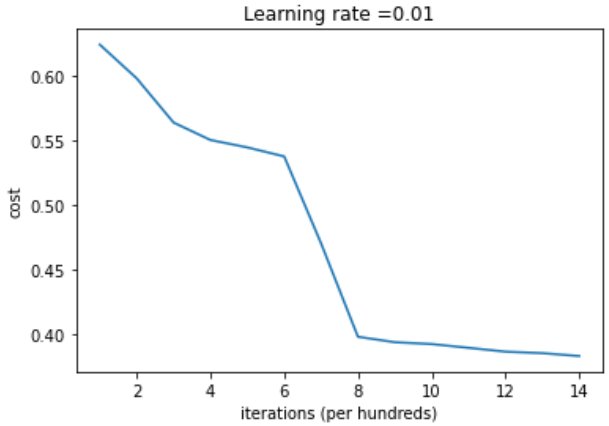

```
On the train set:
Accuracy: 0.5
On the test set:
Accuracy: 0.5
```

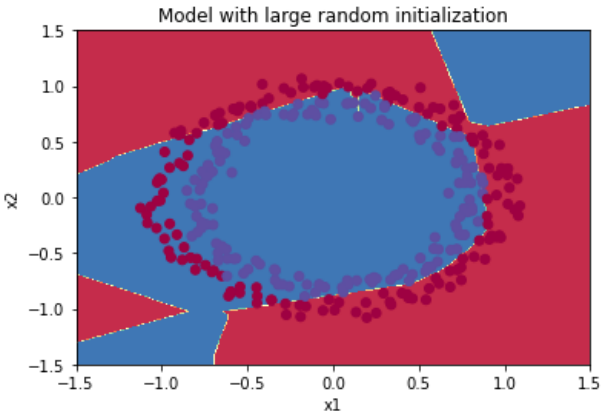
將每一個神經元的權重設為 0 這樣等於沒有更新，每一層輸出都相同，back propagation 神經也是相同的行為，cost 當然不會有改變。

```python
def initialize_parameters_random(layers_dims):
    """
    Arguments:
    layer_dims -- python array (list) containing the size of each layer.

    Returns:
    parameters -- python dictionary containing your parameters "W1", "b1", ..., "WL", "bL":
                    W1 -- weight matrix of shape (layers_dims[1], layers_dims[0])
                    b1 -- bias vector of shape (layers_dims[1], 1)
                    ...
                    WL -- weight matrix of shape (layers_dims[L], layers_dims[L-1])
                    bL -- bias vector of shape (layers_dims[L], 1)
    """

    np.random.seed(3)               # This seed makes sure your "random" numbers will be the as ours
    parameters = {}
    L = len(layers_dims)            # integer representing the number of layers

    for l in range(1, L):
        ### START CODE HERE ### (≈ 2 lines of code)
        parameters['W' + str(l)] = np.random.randn(layers_dims[l], layers_dims[l-1])*10
        parameters['b' + str(l)] = np.zeros((layers_dims[l],1))
        ### END CODE HERE ###

    return parameters
```

```
Cost after iteration 0: inf
Cost after iteration 1000: 0.6239567039908781
Cost after iteration 2000: 0.5978043872838292
Cost after iteration 3000: 0.563595830364618
Cost after iteration 4000: 0.5500816882570866
Cost after iteration 5000: 0.5443417928662615
Cost after iteration 6000: 0.5373553777823036
Cost after iteration 7000: 0.4700141958024487
Cost after iteration 8000: 0.3976617665785177
Cost after iteration 9000: 0.39344405717719166
Cost after iteration 10000: 0.39201765232720626
Cost after iteration 11000: 0.38910685278803786
Cost after iteration 12000: 0.38612995897697244
Cost after iteration 13000: 0.3849735792031832
Cost after iteration 14000: 0.38275100578285265
```

```
W1 = [[ 17.88628473   4.36509851   0.96497468]
 [-18.63492703  -2.77388203  -3.54758979]]
b1 = [[0.]
 [0.]]
W2 = [[-0.82741481 -6.27000677]]
b2 = [[0.]]
```



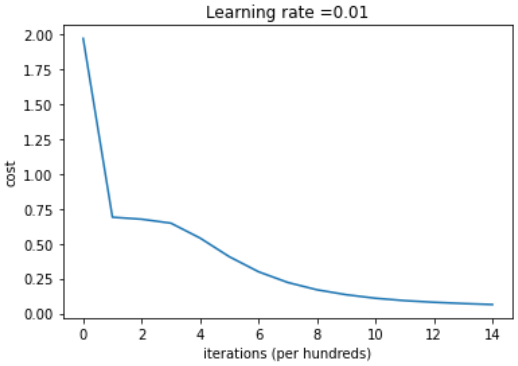



```
On the train set:
Accuracy: 0.83
On the test set:
Accuracy: 0.86
```

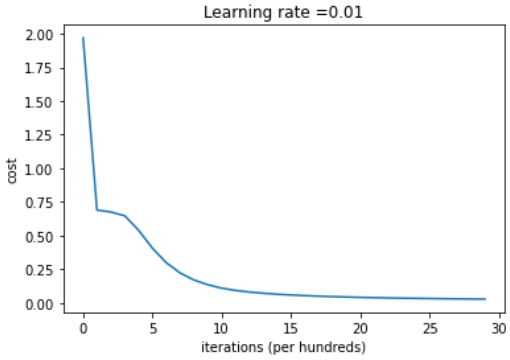
隨機的初始值能避免因對稱性資料而無法梯度下降的問題。一開始出現的 inf 是因為初始值大，激活函數 sigmoid 輸出 0，而使 log(0)=inf。

將初始值修正為 1 倍：

```
Cost after iteration 14000: 0.06370209022580517
```

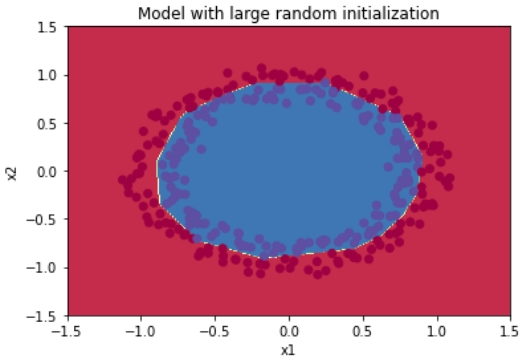



```
On the train set:
Accuracy: 0.9966666666666667
On the test set:
Accuracy: 0.96
```

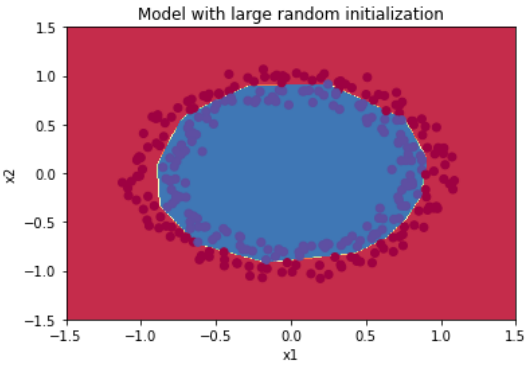



```
On the train set:
Accuracy: 0.9966666666666667
On the test set:
Accuracy: 0.94
```

(iteration=15000)



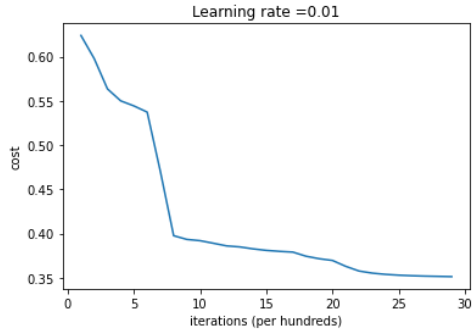
(iteration=30000)



可以見到擬合過好，測試資料正確率也夠高，此時不必再增加學習次數，實際增加學習次數也驗證了效果有限，若要改善 overfitting 應加入 regularization。

將 iteration 提高至 30000，初始值為 10 倍：

```
Cost after iteration 29000: 0.35137288349986656
```



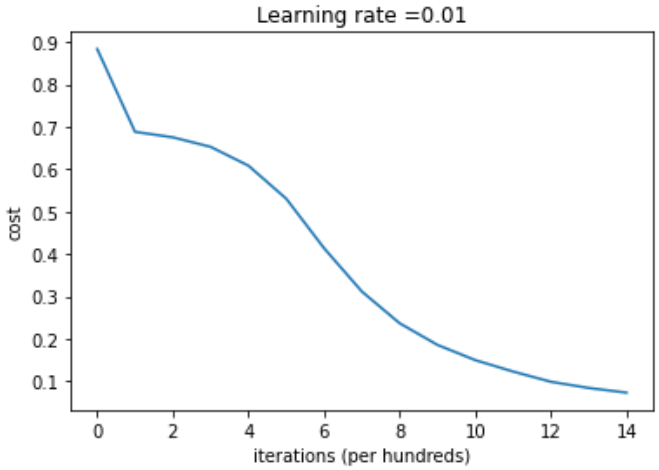

```
On the train set:
Accuracy: 0.85
On the test set:
Accuracy: 0.84
```

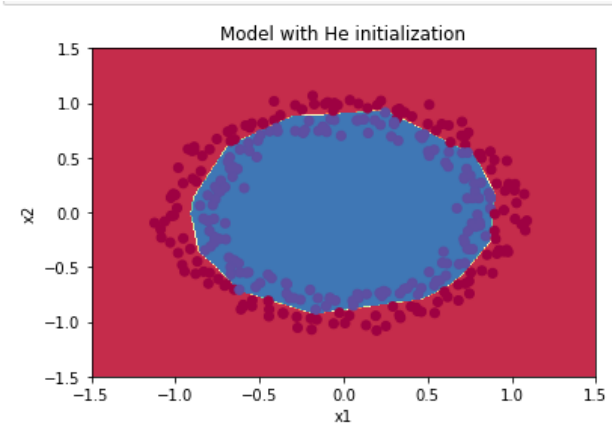
可以看到後期 cost 下降緩慢，訓練資料正確率提高，但測試資料正確率降低。初始值過大，正確率無法像 1 倍初始值如此高。

```python
def initialize_parameters_he(layers_dims):
    """
    Arguments:
    layer_dims -- python array (list) containing the size of each layer.

    Returns:
    parameters -- python dictionary containing your parameters "W1", "b1", ..., "WL", "bL":
                    W1 -- weight matrix of shape (layers_dims[1], layers_dims[0])
                    b1 -- bias vector of shape (layers_dims[1], 1)
                    ...
                    WL -- weight matrix of shape (layers_dims[L], layers_dims[L-1])
                    bL -- bias vector of shape (layers_dims[L], 1)
    """

    np.random.seed(3)
    parameters = {}
    L = len(layers_dims) - 1 # integer representing the number of layers

    for l in range(1, L + 1):
        ### START CODE HERE ### (≈ 2 lines of code)
        parameters['W' + str(l)] = np.random.randn(layers_dims[l], layers_dims[l-1])*np.sqrt(2/layers_dims[l-1])
        parameters['b' + str(l)] = np.zeros((layers_dims[l],1))
        ### END CODE HERE ###

    return parameters
```

```
Cost after iteration 0: 0.8830537463419761
Cost after iteration 1000: 0.6879825919728063
Cost after iteration 2000: 0.6751286264523371
Cost after iteration 3000: 0.6526117768893807
Cost after iteration 4000: 0.6082958970572938
Cost after iteration 5000: 0.5304944491717495
Cost after iteration 6000: 0.4138645817071794
Cost after iteration 7000: 0.3117803464844441
Cost after iteration 8000: 0.23696215330322562
Cost after iteration 9000: 0.18597287209206836
Cost after iteration 10000: 0.15015556280371817
Cost after iteration 11000: 0.12325079292273552
Cost after iteration 12000: 0.09917746546525932
Cost after iteration 13000: 0.08457055954024274
Cost after iteration 14000: 0.07357895962677362
```

```
W1 = [[ 1.78862847  0.43650985]
 [ 0.09649747 -1.8634927 ]
 [-0.2773882  -0.35475898]
 [-0.08274148 -0.62700068]]
b1 = [[0.]
 [0.]
 [0.]
 [0.]]
W2 = [[-0.03098412 -0.33744411 -0.92904268  0.62552248]]
b2 = [[0.]]
```






```
On the train set:
Accuracy: 0.9933333333333333
On the test set:
Accuracy: 0.96
```

與 xavier 不同的是多除以 2，使 variance 不變，並且避免輸出值趨於 0。

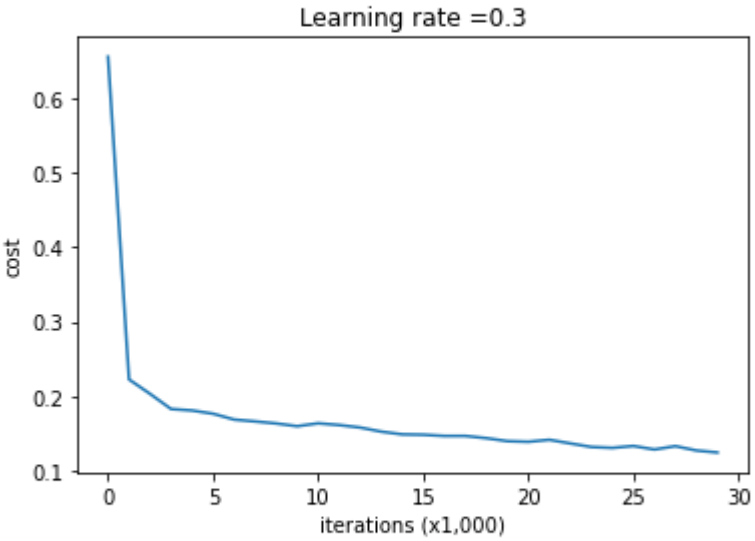
更多的初始值選用方法可以參考：

https://zhuanlan.zhihu.com/p/25110150

Regularization

without regularization：

```
Cost after iteration 0: 0.6557412523481002
Cost after iteration 10000: 0.16329987525724213
Cost after iteration 20000: 0.13851642423245572
```




```
On the training set:
Accuracy: 0.9478672985781991
On the test set:
Accuracy: 0.915
```

with L2：

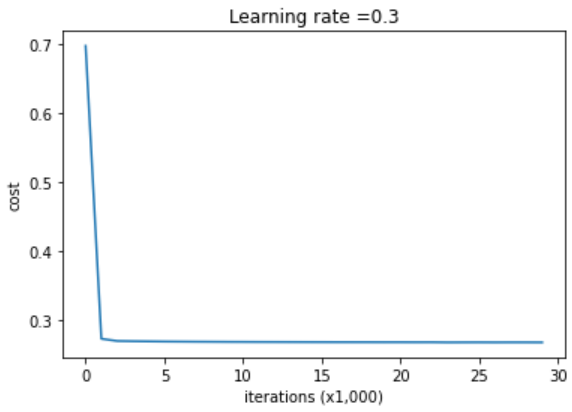
```
def compute_cost_with_regularization(A3, Y, parameters, lambd):
    """
    Implement the cost function with L2 regularization. See formula (2) above.

    Arguments:
    A3 -- post-activation, output of forward propagation, of shape (output size, number of examples)
    Y -- "true" labels vector, of shape (output size, number of examples)
    parameters -- python dictionary containing parameters of the model

    Returns:
    cost - value of the regularized loss function (formula (2))
    """
    m = Y.shape[1]
    W1 = parameters["W1"]
    W2 = parameters["W2"]
    W3 = parameters["W3"]
    cross_entropy_cost = compute_cost(A3, Y) # This gives you the cross-entropy part of the cost

    ### START CODE HERE ### (approx. 1 line)
    L2_regularization_cost = (np.sum(np.square(W1))+np.sum(np.square(W2))+np.sum(np.square(W3)))*(lambd)/(2*m)
    ### END CODER HERE ###

    cost = cross_entropy_cost + L2_regularization_cost

    return cost
```

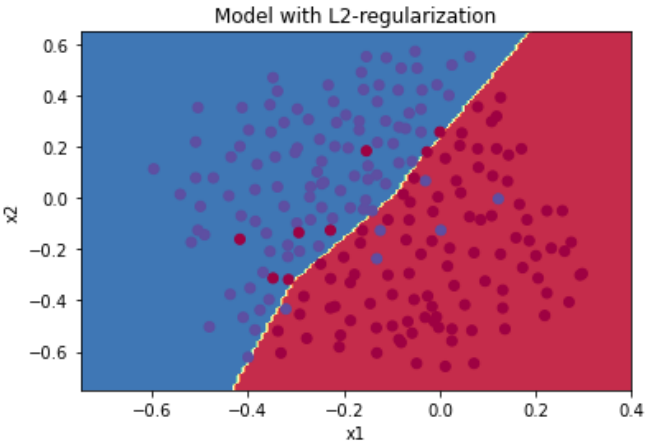
```
cost = 1.7864859451590758
```

```python
def backward_propagation_with_regularization(X, Y, cache, lambd):
    """
    Implements the backward propagation of our baseline model to which we added an L2 regularization.

    Arguments:
    X -- input dataset, of shape (input size, number of examples)
    Y -- "true" labels vector, of shape (output size, number of examples)
    cache -- cache output from forward_propagation()
    lambd -- regularization hyperparameter, scalar

    Returns:
    gradients -- A dictionary with the gradients with respect to each parameter, activation and pre-activation variables
    """

    m = X.shape[1]
    (Z1, A1, W1, b1, Z2, A2, W2, b2, Z3, A3, W3, b3) = cache

    dZ3 = A3 - Y

    ### START CODE HERE ### (approx. 1 line)
    dW3 = 1./m * np.dot(dZ3, A2.T) + lambd*W3/m
    ### END CODE HERE ###
    db3 = 1./m * np.sum(dZ3, axis=1, keepdims = True)

    dA2 = np.dot(W3.T, dZ3)
    dZ2 = np.multiply(dA2, np.int64(A2 > 0))
    ### START CODE HERE ### (approx. 1 line)
    dW2 = 1./m * np.dot(dZ2, A1.T) + lambd*W2/m
    ### END CODE HERE ###
    db2 = 1./m * np.sum(dZ2, axis=1, keepdims = True)

    dA1 = np.dot(W2.T, dZ2)
    dZ1 = np.multiply(dA1, np.int64(A1 > 0))
    ### START CODE HERE ### (approx. 1 line)
    dW1 = 1./m * np.dot(dZ1, X.T) + lambd*W1/m
    ### END CODE HERE ###
    db1 = 1./m * np.sum(dZ1, axis=1, keepdims = True)

    gradients = {"dZ3": dZ3, "dW3": dW3, "db3": db3,"dA2": dA2,
                 "dZ2": dZ2, "dW2": dW2, "db2": db2, "dA1": dA1,
                 "dZ1": dZ1, "dW1": dW1, "db1": db1}

    return gradients
```

```
dW1 = [[-0.25604646  0.12298827 -0.28297129]
 [-0.17706303  0.34536094 -0.4410571 ]]
dW2 = [[ 0.79276486  0.85133918]
 [-0.0957219  -0.01720463]
 [-0.13100772 -0.03750433]]
dW3 = [[-1.77691347 -0.11832879 -0.09397446]]
```

```
Cost after iteration 0: 0.6974484493131264
Cost after iteration 10000: 0.2684918873282239
Cost after iteration 20000: 0.2680916337127301
```






```
On the train set:
Accuracy: 0.9383886255924171
On the test set:
Accuracy: 0.93
```

加入 L2 逞罰權重，可以看到訓練資料的正確率下降，改善 overfitting 的問題，由圖可發現經過 1000 次學習 cost 就收斂。

with dropout：

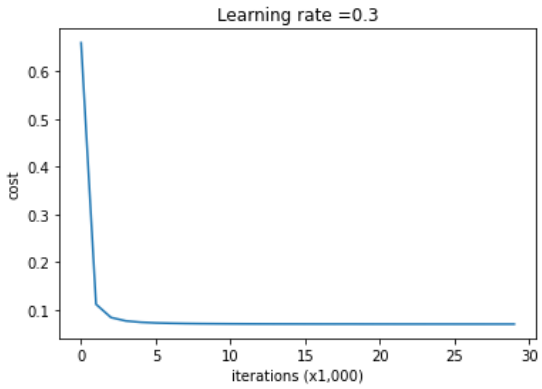
```
def forward_propagation_with_dropout(X, parameters, keep_prob = 0.5):

    np.random.seed(1)

    # retrieve parameters
    W1 = parameters["W1"]
    b1 = parameters["b1"]
    W2 = parameters["W2"]
    b2 = parameters["b2"]
    W3 = parameters["W3"]
    b3 = parameters["b3"]

    # LINEAR -> RELU -> LINEAR -> RELU -> LINEAR -> SIGMOID
    Z1 = np.dot(W1, X) + b1
    A1 = relu(Z1)
    ### START CODE HERE ### (approx. 4 lines)         # Steps 1-4 below correspond
    D1 = np.random.randn(*A1.shape)     # Step 1: initialize matrix D1 = np.random.
    D1 = D1<keep_prob                                 # Step 2: convert entr
    A1 = np.multiply(A1,D1)                               # Step 3: shut
    A1 = A1/keep_prob                                 # Step 4: scale the va
    ### END CODE HERE ###
    Z2 = np.dot(W2, A1) + b2
    A2 = relu(Z2)
    ### START CODE HERE ### (approx. 4 lines)
    D2 = np.random.randn(*A2.shape)       # Step 1: initialize matrix D2 = np.rand
    D2 = D2<keep_prob                                  # Step 2: convert ent
    A2 = np.multiply(A2,D2)                                # Step 3: shut d
    A2 = A2/keep_prob                                  # Step 4: scale the v
    ### END CODE HERE ###
    Z3 = np.dot(W3, A2) + b3
    A3 = sigmoid(Z3)

    cache = (Z1, D1, A1, W1, b1, Z2, D2, A2, W2, b2, Z3, A3, W3, b3)

    return A3, cache
```

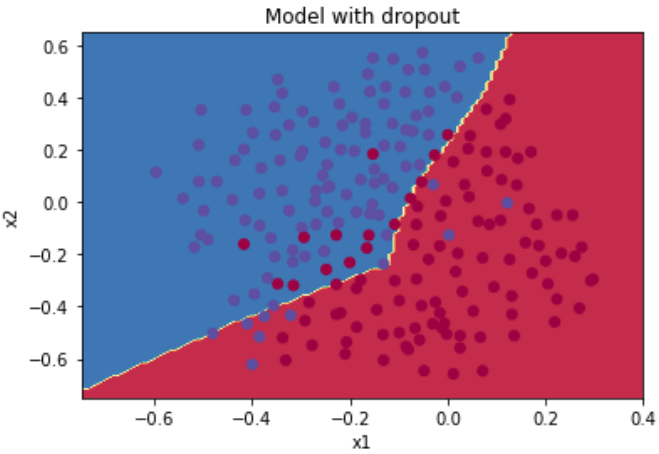
```
A3 = [[0.49683389 0.05332327 0.04565099 0.01446893 0.49683389]]
```

```
def backward_propagation_with_dropout(X, Y, cache, keep_prob):

    m = X.shape[1]
    (Z1, D1, A1, W1, b1, Z2, D2, A2, W2, b2, Z3, A3, W3, b3) = cache

    dZ3 = A3 - Y
    dW3 = 1./m * np.dot(dZ3, A2.T)
    db3 = 1./m * np.sum(dZ3, axis=1, keepdims = True)
    dA2 = np.dot(W3.T, dZ3)
    ### START CODE HERE ### (≈ 2 lines of code)
    dA2 = dA2*D2              # Step 1: Apply mask D2 to shut down the same neurons
    dA2 = dA2/keep_prob              # Step 2: Scale the value of neurons that haven
    ### END CODE HERE ###
    dZ2 = np.multiply(dA2, np.int64(A2 > 0))
    dW2 = 1./m * np.dot(dZ2, A1.T)
    db2 = 1./m * np.sum(dZ2, axis=1, keepdims = True)

    dA1 = np.dot(W2.T, dZ2)
    ### START CODE HERE ### (≈ 2 lines of code)
    dA1 = dA1*D1              # Step 1: Apply mask D1 to shut down the same neurons
    dA1 = dA1/keep_prob              # Step 2: Scale the value of neurons that haven
    ### END CODE HERE ###
    dZ1 = np.multiply(dA1, np.int64(A1 > 0))
    dW1 = 1./m * np.dot(dZ1, X.T)
    db1 = 1./m * np.sum(dZ1, axis=1, keepdims = True)

    gradients = {"dZ3": dZ3, "dW3": dW3, "db3": db3,"dA2": dA2,
                 "dZ2": dZ2, "dW2": dW2, "db2": db2, "dA1": dA1,
                 "dZ1": dZ1, "dW1": dW1, "db1": db1}

    return gradients
```

```
dA1 = [[ 0.36544439  0.         -0.00188233  0.         -0.17408748]
 [ 0.65515713  0.         -0.00337459  0.         -0.         ]]
dA2 = [[ 0.58180856  0.         -0.00299679  0.         -0.27715731]
 [ 0.          0.53159854 -0.          0.53159854 -0.34089673]
 [ 0.          0.         -0.00292733  0.         -0.         ]]
```

```
Cost after iteration 10000: 0.07083004396279081
Cost after iteration 20000: 0.07030667016479066
```

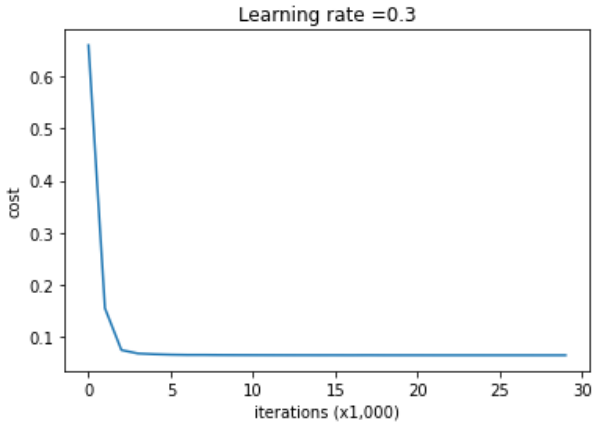


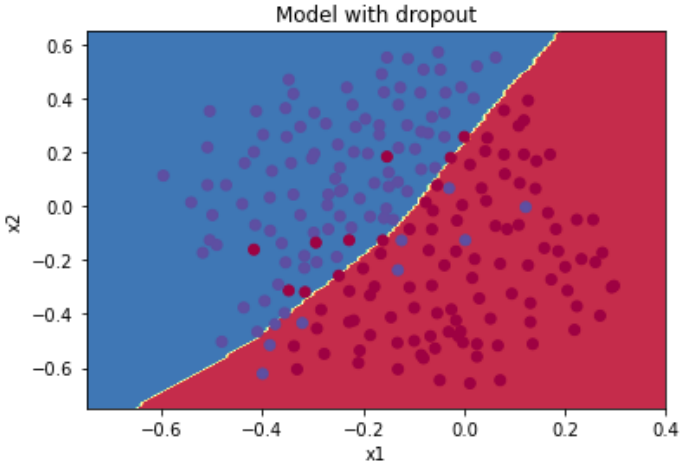



```
On the train set:
Accuracy: 0.8957345971563981
On the test set:
Accuracy: 0.89
```

在 A3 的部分與 expected value 有差，推測是 seed 不同造成的，因此出來的正確率 89%也與 95%有所差距，嘗試將替換 seed 為 seed(2)：

```
Cost after iteration 0: 0.6596006403732521
Cost after iteration 10000: 0.06561489978266935
Cost after iteration 20000: 0.06542385125907116
```



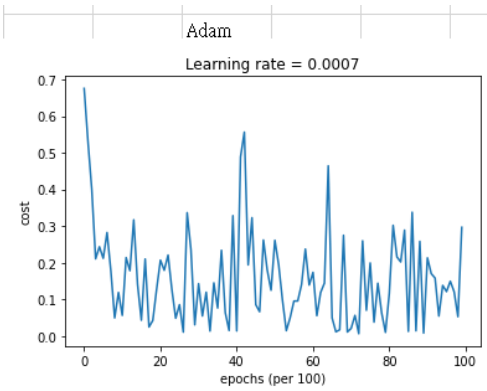



```
On the train set:
Accuracy: 0.9241706161137441
On the test set:
Accuracy: 0.925
```

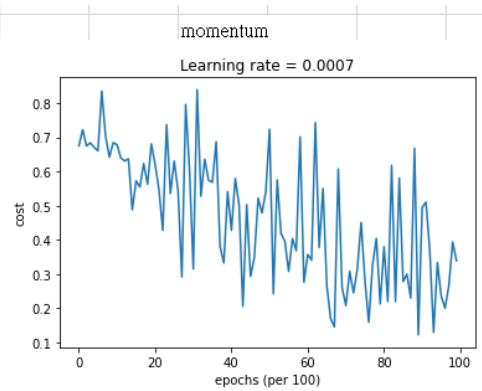
獲得更高的正確率，以及 boundary 更為平滑，經由更改隨機權重，調整為更好的模型。

在上次作業中可以知道改變 minibatch 的大小會影響到正確率。

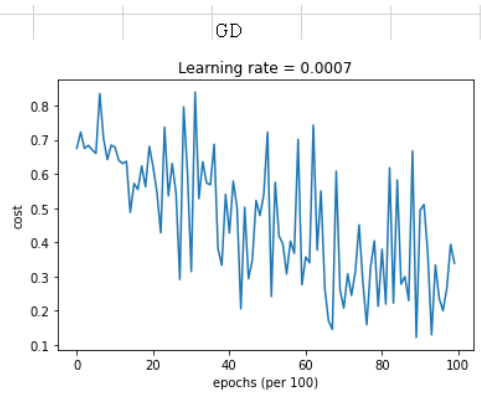
minibatch=32



Accuracy: 0.94

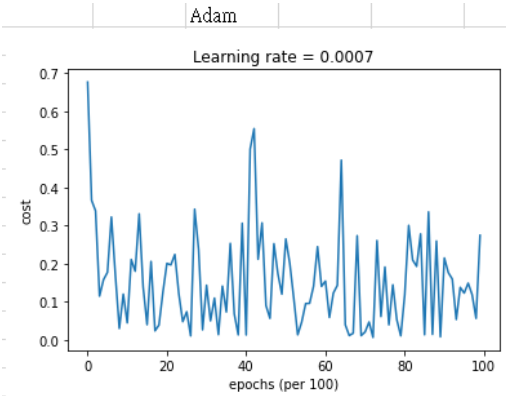


Accuracy: 0.8766666666666667

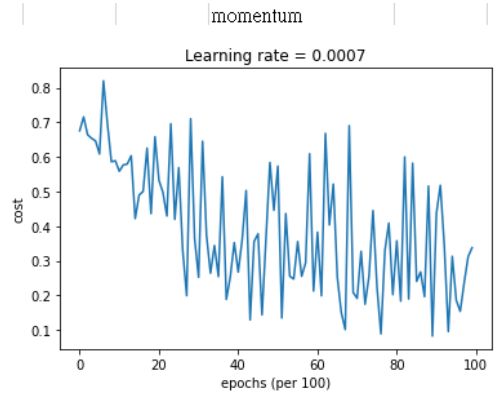


Accuracy: 0.8766666666666667

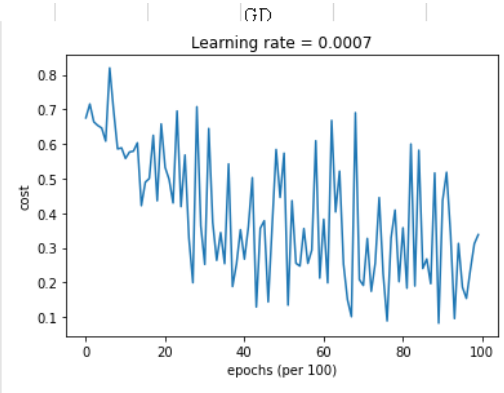
minibatch=32



Accuracy: 0.94



Accuracy: 0.9133333333333333



Accuracy: 0.9133333333333333

將以上測試套用 HW6

由於當時 HW6 為了提高正確率而修改許多參數

num_epochs、learning_rate、增加一層 conV2D

因此接下來的測試結果預設的可能和當初 HW6 給的有所不同

先還原 HW6 最一開始網大 PDF 給的數據

minibatch_size = 64， num_epochs = 100，learning_rate = 0.009，兩層網路

再慢慢加入 dropout 以及 L2 regularization

增加一層 conV2D

再加入 dropout 以及 L2 regularization

比較正確率

(一) minibatch_size = 64， num_epochs = 100，learning_rate = 0.009，兩層網路 keep_prob 和以後要用到的第三層網路先以#註記不使用

```
def forward_propagation(X, parameters,keep_prob):
    """
    Implements the forward propagation for the model:
    CONV2D -> RELU -> MAXPOOL -> CONV2D -> RELU -> MAXPOOL -> FLATTEN -> FULLYCONNECTED

    Arguments:
    X -- input dataset placeholder, of shape (input size, number of examples)
    parameters -- python dictionary containing your parameters "W1", "W2"
                  the shapes are given in initialize_parameters

    Returns:
    Z3 -- the output of the last LINEAR unit
    """

    # Retrieve the parameters from the dictionary "parameters"
    W1 = parameters['W1']
    W2 = parameters['W2']
    #W3 = parameters['W3']
    ### START CODE HERE ###
    # CONV2D: stride of 1, padding 'SAME'
    Z1 =  tf.nn.conv2d(X,filter=W1,strides=(1,1,1,1),padding="SAME")
    # RELU
    A1 =  tf.nn.relu(Z1)
    #A1=tf.nn.dropout(A1,keep_prob)
    # MAXPOOL: window 8x8, sride 8, padding 'SAME'
    P1 =  tf.nn.max_pool(A1,ksize=(1,8,8,1),strides=(1,8,8,1),padding='SAME')
    # CONV2D: filters W2, stride 1, padding 'SAME'
    Z2 =  tf.nn.conv2d(P1,filter=W2,strides=(1,1,1,1),padding="SAME")
    # RELU
    A2 =  tf.nn.relu(Z2)
    #A2=tf.nn.dropout(A2,keep_prob)
    # MAXPOOL: window 4x4, stride 4, padding 'SAME'
    P2 =  tf.nn.max_pool(A2,ksize=(1,4,4,1),strides=(1,4,4,1),padding="SAME")
    #Z3 =  tf.nn.conv2d(P2,filter=W3,strides=(1,1,1,1),padding="SAME")
    #A3 =  tf.nn.relu(Z3)
    #P3 =  tf.nn.max_pool(A3,ksize=(1,2,2,1),strides=(1,2,2,1),padding="SAME")
    # FLATTEN
    F =  tf.contrib.layers.flatten(P2)
    # FULLY-CONNECTED without non-linear activation function (not not call softmax).
    # 6 neurons in output layer. Hint: one of the arguments should be "activation_fn=None"
    Z3 =  tf.contrib.layers.fully_connected(F,activation_fn=None,num_outputs=6)
    ### END CODE HERE ###

    return Z3
```

```
Cost after epoch 100: 0.156213
Train Accuracy: 0.9768519
Test Accuracy: 0.85
```

在不使用 regularization 的情況正確率就有 85%，訓練資料也沒有過度擬合。

(二) minibatch_size = 64， num_epochs = 100，learning_rate = 0.009，兩層網路將 keep_prob 設定為 0.5，dropout 註記刪除，加入 dropout

```
def forward_propagation(X, parameters,keep_prob):
    """
    Implements the forward propagation for the model:
    CONV2D -> RELU -> MAXPOOL -> CONV2D -> RELU -> MAXPOOL -> FLATTEN -> FULLYCONNECTED

    Arguments:
    X -- input dataset placeholder, of shape (input size, number of examples)
    parameters -- python dictionary containing your parameters "W1", "W2"
                  the shapes are given in initialize_parameters

    Returns:
    Z3 -- the output of the last LINEAR unit
    """

    # Retrieve the parameters from the dictionary "parameters"
    W1 = parameters['W1']
    W2 = parameters['W2']
    #W3 = parameters['W3']
    ### START CODE HERE ###
    # CONV2D: stride of 1, padding 'SAME'
    Z1 =  tf.nn.conv2d(X,filter=W1,strides=(1,1,1,1),padding="SAME")
    # RELU
    A1 =  tf.nn.relu(Z1)
    A1=tf.nn.dropout(A1,keep_prob)
    # MAXPOOL: window 8x8, sride 8, padding 'SAME'
    P1 =  tf.nn.max_pool(A1,ksize=(1,8,8,1),strides=(1,8,8,1),padding='SAME')
    # CONV2D: filters W2, stride 1, padding 'SAME'
    Z2 =  tf.nn.conv2d(P1,filter=W2,strides=(1,1,1,1),padding="SAME")
    # RELU
    A2 =  tf.nn.relu(Z2)
    A2=tf.nn.dropout(A2,keep_prob)
    # MAXPOOL: window 4x4, stride 4, padding 'SAME'
    P2 =  tf.nn.max_pool(A2,ksize=(1,4,4,1),strides=(1,4,4,1),padding="SAME")
    #Z3 =  tf.nn.conv2d(P2,filter=W3,strides=(1,1,1,1),padding="SAME")
    #A3 =  tf.nn.relu(Z3)
    #P3 =  tf.nn.max_pool(A3,ksize=(1,2,2,1),strides=(1,2,2,1),padding="SAME")
    # FLATTEN
    F =  tf.contrib.layers.flatten(P2)
    # FULLY-CONNECTED without non-linear activation function (not not call softmax).
    # 6 neurons in output layer. Hint: one of the arguments should be "activation_fn=None"
    Z3 =  tf.contrib.layers.fully_connected(F,activation_fn=None,num_outputs=6)
    ### END CODE HERE ###

    return Z3
```

```
Cost after epoch 100: 0.740163
Train Accuracy: 0.75277776
Test Accuracy: 0.675
```

隨機關閉一半的神經元，但 epoch 只有 100 次，使得特徵抓取不足，正確率很低。

(三) minibatch_size = 64， num_epochs = 100，learning_rate = 0.009，兩層網路取消 dropout，採用 L2 regularization， lambd=0.1

```python
def compute_cost(Z3, Y,parameters, lambd):
    """
    Computes the cost

    Arguments:
    Z3 -- output of forward propagation (output of the last LINEAR unit), of shape (6, number of examples)
    Y -- "true" labels vector placeholder, same shape as Z3

    Returns:
    cost - Tensor of the cost function
    """
    m = Y.shape[1]
    W1 = parameters["W1"]
    W2 = parameters["W2"]
    W3 = parameters["W3"]
    regularizer = tf.nn.l2_loss(W1)+tf.nn.l2_loss(W2)#+tf.nn.l2_loss(W3)
    ### START CODE HERE ### (1 line of code)
    cost =  tf.reduce_mean(tf.nn.softmax_cross_entropy_with_logits(logits=Z3,labels=Y))
    ### END CODE HERE ###
    cost = tf.reduce_mean(cost + lambd * regularizer)
    return cost
```

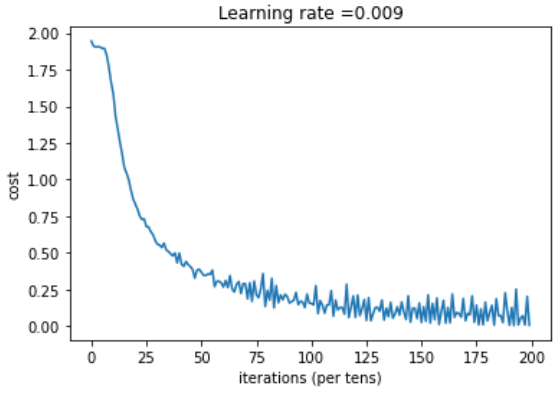
```
Cost after epoch 100: 1.546288
Train Accuracy: 0.5574074
Test Accuracy: 0.51666665
```

由於原本的訓練資料正確率還沒出現過度擬合，現又逞罰權重，使訓練緩慢，需要增加更多的 epoch 正確率才會提高。

以上 regularization 需要更多的 epoch 才能增進模型效能，將 epoch 提高至 200 次。由於電腦效能較差，理應至少 500 次才會有好表現，待之後作業時測。

(一) minibatch_size = 64， num_epochs = 200，learning_rate = 0.009，兩層網路 keep_prob 和以後要用到的第三層網路先以#註記不使用

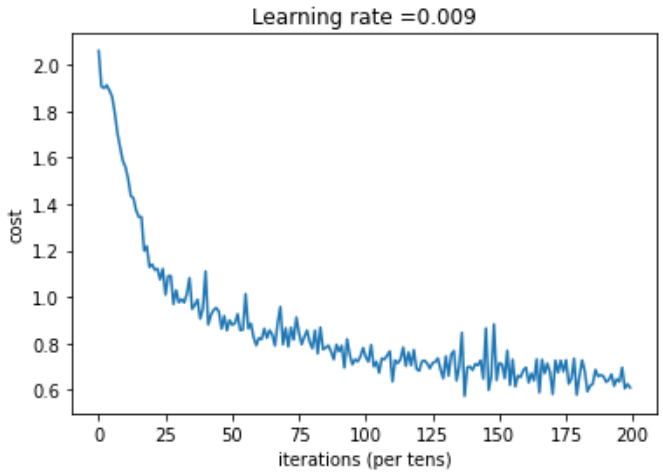
```
Cost after epoch 175: 0.144362
Train Accuracy: 1.0
Test Accuracy: 0.8666667
Cost after epoch 180: 0.037744
Train Accuracy: 0.8138889
Test Accuracy: 0.7583333
Cost after epoch 185: 0.072118
Train Accuracy: 0.9425926
Test Accuracy: 0.8333333
Cost after epoch 190: 0.007717
Train Accuracy: 0.9777778
Test Accuracy: 0.85
Cost after epoch 195: 0.055194
Train Accuracy: 0.8472222
Test Accuracy: 0.76666665
```



正確率最高出現在第 175 次，有 86.7%，與 epoch=100 相差無幾，呈現收斂趨勢。

(二) minibatch_size = 64， num_epochs = 200，learning_rate = 0.009，兩層網路將 keep_prob 設定為 0.5，dropout 註記刪除，加入 dropout

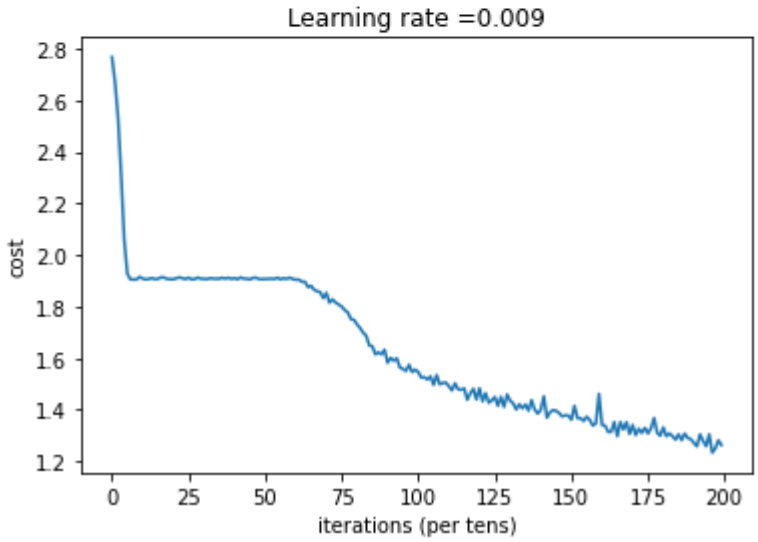
```
Cost after epoch 165: 0.587442
Train Accuracy: 0.7787037
Test Accuracy: 0.81666666
Cost after epoch 170: 0.580631
Train Accuracy: 0.7925926
Test Accuracy: 0.73333335
Cost after epoch 175: 0.727840
Train Accuracy: 0.78981483
Test Accuracy: 0.8
Cost after epoch 180: 0.657123
Train Accuracy: 0.78425926
Test Accuracy: 0.775
Cost after epoch 185: 0.625394
Train Accuracy: 0.8055556
Test Accuracy: 0.78333336
Cost after epoch 190: 0.633256
Train Accuracy: 0.74814814
Test Accuracy: 0.73333335
Cost after epoch 195: 0.634317
Train Accuracy: 0.8055556
Test Accuracy: 0.725
```



正確率最高出現在第 165 次，有 81.7%，和 epoch=100 的 67.5%正確率相比提高了許多，但也可以發現 165 次以後正確率震盪明顯，顯示再提高 epcoh 效果應不佳，得換初始值或改變關閉的神經元。

(三) minibatch_size = 64， num_epochs = 200，learning_rate = 0.009，兩層網路取消 dropout，採用 L2 regularization，lambd=0.1

```
Cost after epoch 170: 1.340957
Train Accuracy: 0.65185183
Test Accuracy: 0.625
Cost after epoch 175: 1.308055
Train Accuracy: 0.5842593
Test Accuracy: 0.56666666
Cost after epoch 180: 1.331653
Train Accuracy: 0.64537036
Test Accuracy: 0.6166667
Cost after epoch 185: 1.304852
Train Accuracy: 0.6388889
Test Accuracy: 0.60833335
Cost after epoch 190: 1.271138
Train Accuracy: 0.6157407
Test Accuracy: 0.55833334
Cost after epoch 195: 1.304764
Train Accuracy: 0.65092593
Test Accuracy: 0.60833335
```



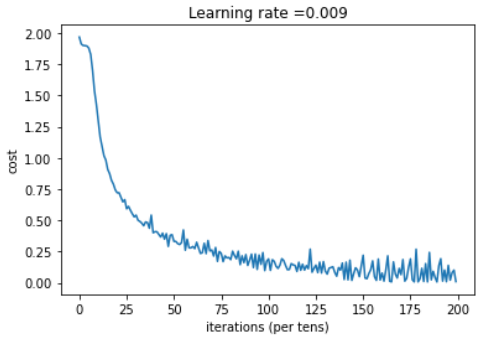
正確率最高出現在第 170 次，有 62.5%，和 epoch=100 的 51.7%正確率相比提高了許多，但也可以發現 170 次以後正確率震盪明顯，顯示再提高 epcoh 效果應不佳，得換初始值或改變 lambd 值。

增加一層 Conv2D

```python
def forward_propagation(X, parameters,keep_prob):
    """
    Implements the forward propagation for the model:
    CONV2D -> RELU -> MAXPOOL -> CONV2D -> RELU -> MAXPOOL -> FLATTEN -> FULLYCONNECTED

    Arguments:
    X -- input dataset placeholder, of shape (input size, number of examples)
    parameters -- python dictionary containing your parameters "W1", "W2"
                  the shapes are given in initialize_parameters

    Returns:
    Z3 -- the output of the last LINEAR unit
    """

    # Retrieve the parameters from the dictionary "parameters"
    W1 = parameters['W1']
    W2 = parameters['W2']
    #W3 = parameters['W3']
    ### START CODE HERE ###
    # CONV2D: stride of 1, padding 'SAME'
    Z1 =  tf.nn.conv2d(X,filter=W1,strides=(1,1,1,1),padding="SAME")
    # RELU
    A1 =  tf.nn.relu(Z1)
    #A1=tf.nn.dropout(A1,keep_prob)
    # MAXPOOL: window 8x8, sride 8, padding 'SAME'
    P1 =  tf.nn.max_pool(A1,ksize=(1,8,8,1),strides=(1,8,8,1),padding='SAME')
    # CONV2D: filters W2, stride 1, padding 'SAME'
    Z2 =  tf.nn.conv2d(P1,filter=W2,strides=(1,1,1,1),padding="SAME")
    # RELU
    A2 =  tf.nn.relu(Z2)
    #A2=tf.nn.dropout(A2,keep_prob)
    # MAXPOOL: window 4x4, stride 4, padding 'SAME'
    P2 =  tf.nn.max_pool(A2,ksize=(1,4,4,1),strides=(1,4,4,1),padding="SAME")
    Z3 =  tf.nn.conv2d(P2,filter=W3,strides=(1,1,1,1),padding="SAME")
    A3 =  tf.nn.relu(Z3)
    #A3=tf.nn.dropout(A3,keep_prob)
    P3 =  tf.nn.max_pool(A3,ksize=(1,2,2,1),strides=(1,2,2,1),padding="SAME")
    # FLATTEN
    F =  tf.contrib.layers.flatten(P2)
    # FULLY-CONNECTED without non-linear activation function (not not call softmax).
    # 6 neurons in output layer. Hint: one of the arguments should be "activation_fn=None"
    Z3 =  tf.contrib.layers.fully_connected(F,activation_fn=None,num_outputs=6)
    ### END CODE HERE ###

    return Z3
```

(一) minibatch_size = 64， num_epochs = 200，learning_rate = 0.009，三層網路 keep_prob 先以#註記不使用

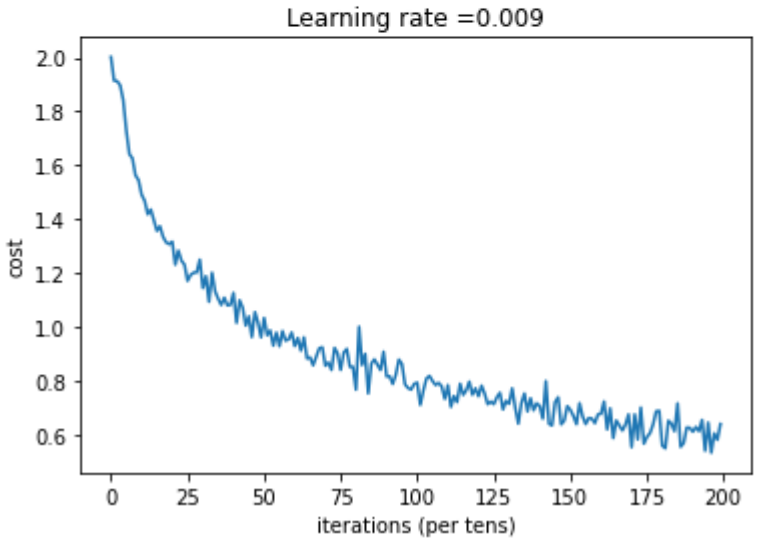
```
Cost after epoch 185: 0.243760
Train Accuracy: 0.98425925
Test Accuracy: 0.9
Cost after epoch 190: 0.137130
Train Accuracy: 0.99351853
Test Accuracy: 0.875
Cost after epoch 195: 0.139968
Train Accuracy: 0.9925926
Test Accuracy: 0.8666667
```



正確率最高出現在第 185 次，有 90%，與兩層的相比提高了 3%左右，這表示增加網路確實能改善正確率。

(二) minibatch_size = 64， num_epochs = 200，learning_rate = 0.009，兩層網路將 keep_prob 設定為 0.5，dropout 註記刪除，加入 dropout

```
Cost after epoch 180: 0.559167
Train Accuracy: 0.83148146
Test Accuracy: 0.80833334
Cost after epoch 185: 0.716077
Train Accuracy: 0.8064815
Test Accuracy: 0.84166664
Cost after epoch 190: 0.610826
Train Accuracy: 0.7537037
Test Accuracy: 0.69166666
Cost after epoch 195: 0.644541
Train Accuracy: 0.8296296
Test Accuracy: 0.8
```

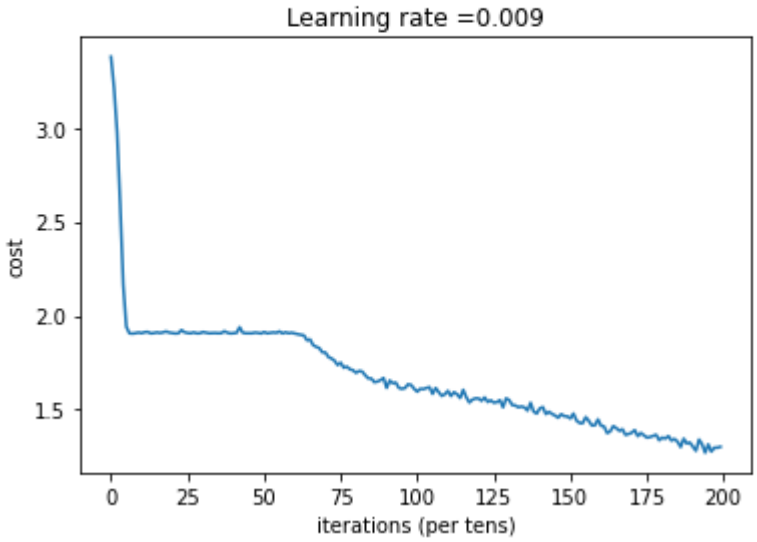


正確率最高出現在第 185 次，有 84.2%，與兩層的相比提高了 3%左右，這表示增加網路確實能改善正確率，且觀察 cost 還能繼續下降，增加 epoch 理論上能再提高正確率。

(三) minibatch_size = 64， num_epochs = 200，learning_rate = 0.009，兩層網路取消 dropout，採用 L2 regularization

```
def compute_cost(Z3, Y,parameters, lambd):
    """
    Computes the cost

    Arguments:
    Z3 -- output of forward propagation (output of the last LINEAR unit), of shape (6, number of examples)
    Y -- "true" labels vector placeholder, same shape as Z3

    Returns:
    cost - Tensor of the cost function
    """
    m = Y.shape[1]
    W1 = parameters["W1"]
    W2 = parameters["W2"]
    W3 = parameters["W3"]
    regularizer = tf.nn.l2_loss(W1)+tf.nn.l2_loss(W2)+tf.nn.l2_loss(W3)
    ### START CODE HERE ### (1 line of code)
    cost =  tf.reduce_mean(tf.nn.softmax_cross_entropy_with_logits(logits=Z3,labels=Y))
    ### END CODE HERE ###
    cost = tf.reduce_mean(cost + lambd * regularizer)
    return cost
```

```
Cost after epoch 195: 1.315452
Train Accuracy: 0.64166665
Test Accuracy: 0.65833336
```



正確率最高出現在第 195 次，有 65.8%，與兩層的相比提高了 3%左右，這表示增加網路確實能改善正確率，且觀察 cost 和正確率並無震盪現像，增加 epoch 一定能再提高正確率。

綜合以上觀察，加入 dropout 若沒有提高 epoch 可能會使網路訓練不完整，少抓許多特徵，導致正確率在相同 epoch 的情況下很低，同理加入 L2 逞罰權重，會使訓練較慢，使相同 epoch 的情況下正確率表現不好，解決方法就是增加 epoch，在 HW8 中 epoch 是以萬為單位，而應用在 HW6 只有百為單位，量級差太多。